

# actinn-jax: a fast, low-memory, CPU-first classifier for reference-based single-cell type annotation, with a large-to-refined workflow

Ian Driver<sup>1,\*</sup>

## Abstract

Reference-based cell-type annotation is a routine, high-volume step in single-cell analysis, and most labs run it on a laptop, not a GPU cluster. We present **actinn-jax**, a from-scratch JAX reimplementation of ACTINN (a 4-layer MLP classifier) with a sparse-aware pipeline and a train-once/map-many cached reference model, and benchmark it neutrally against **nine** other methods spanning every major method class — classical (SVM, kNN), regularized-linear (CellTypist), marker/correlation (SingleR, scmap), the original TensorFlow ACTINN, deep probabilistic (scANVI, scArches), and a foundation model (scPRINT) — across **six** datasets (within-dataset, cross-dataset, and cross-study; 8 to 86 cell types) on commodity Apple-Silicon hardware. We measure accuracy, macro-F1, ontology-aware concordance, training time, inference time, and peak memory. **Across the five datasets that share a label vocabulary between reference and query, actinn-jax ties the most accurate methods (mean accuracy 0.831, vs. scANVI 0.835 and scArches 0.833 — a five-way top cluster within 0.004) while predicting  $\sim 195\times$  faster than scANVI (0.37 s vs. 73 s per query) and using  $3.6\times$  less memory than its own TensorFlow predecessor; no benchmarked method dominates it on both accuracy and speed.** (The sixth dataset, cross-dataset lung, is a label-vocabulary mismatch — see §3.1 — where every method scores  $\sim 0.35$  by exact match but  $\sim 0.75$  by ontology-aware concordance.) The central finding is that actinn-jax is **Pareto-efficient**: it matches the accuracy of the strongest methods while being in the fastest-inference, lightest cluster. **We are explicit that it is not the accuracy champion** — deep methods and boosted trees edge it by 1–2 points where they run, and a benchmark claiming otherwise would be wrong — but no benchmarked method dominates it on *both* accuracy and cost, and its differentiators are practical: sub-second, memory-bounded inference and a **large→refined annotation workflow** that a census-scale broad model ( $\sim 800$  human cell types, with a calibrated abstain) hands off to a small focused model for the tissue at hand (e.g. a 48-type HLiCA liver reference that lifts held-out cross-study accuracy from 0.23/0.58 to 0.72/0.86, exact-CL/ontology), plus within-cell-type resolution (hepatocyte zonation). An **independent external validation** on the community Open Problems **label\_projection** benchmark reproduces the picture: actinn-jax places 3rd of 17 on accuracy (1st among methods that complete every dataset), and a **controlled same-hardware** rerun shows it matching its PCA-space **mlp** sibling at  $\sim 2\times$  the speed and beating xgboost’s accuracy at  $\sim 5.5\times$  less runtime and  $\sim 4\times$  less memory. We report where it loses (small references, very few cells per type, and fine-grained cross-domain targets) and what closes the remaining gaps cheaply (opt-in standardization; a reference-guided gene budget).

<sup>1</sup> Affiliation to be confirmed.

\* Correspondence: driver.ian@gmail.com

# 1 Introduction

Annotating cell types by mapping to a labeled reference is one of the most frequently run operations in single-cell RNA-seq. The method landscape is large (survey), but published comparisons ([Abdelaal 2019], [Huang 2024]) emphasize accuracy and rarely report the axis that decides what a working scientist actually runs on their own machine: **wall-clock time and memory on commodity hardware**, without a GPU. Foundation models (scGPT, Geneformer, scPRINT) push accuracy in some settings but need GPUs and minutes-to- hours, and — as we and others find — their *zero-shot* label predictions underperform a small model trained on curated reference labels.

We ask a focused, practical question: **where does a small, CPU-first classifier land on the accuracy  $\times$  runtime  $\times$  memory frontier**, against the methods people actually use? We answer it with `actinn-jax`, and we hold ourselves to a neutral standard — every method runs in its own environment through the same harness, on the same splits, scored by the same metrics, with the honest result reported even where `actinn-jax` loses.

**Contributions.** 1. A modern, dependency-light (no TensorFlow) JAX reimplementation of ACTINN with sparse preprocessing, chunked atlas-scale prediction, and a cached reference model. 2. A neutral 10-method  $\times$  6-dataset benchmark of accuracy, speed, and memory on Apple Silicon, with scaling curves and a rejection/abstain analysis. 3. A **large $\rightarrow$ refined annotation workflow**: a shipped census-wide  $\sim$ 800-type human reference (with a calibrated abstain) provides a broad first pass, then hands off to a small focused reference for the tissue of interest — the refinement recovers large accuracy on cross-study data where the broad model is weak — plus within-cell-type resolution (hepatocyte zonation) and a foundation-model-guided coarse $\rightarrow$ fine hierarchy. 4. An **independent external validation** on Open Problems `label_projection`, with a controlled same-hardware speed/memory comparison, and a small set of cheap, honest ablations that characterize *when* the gene-space model gains or loses to heavier methods (input standardization; a reference-guided gene budget; a foundation-model negative control).

## 2 Methods

### 2.1 `actinn-jax`

`actinn-jax` reimplements ACTINN ([Ma & Pellegrini 2020]) — a 4-layer fully-connected network (100/50/25 hidden units, ReLU, softmax) trained with Adam — in JAX/optax, replacing the original’s TensorFlow-1.x graph/session code. Key engineering: - **Sparse-aware preprocessing**: no `adata.to_df` densification; CP10k+log2 normalization and the `expr/CV` gene filter run on sparse matrices; only the selected-gene columns of each minibatch are densified. - **Cached reference model** (`ReferenceModel`): train once, `save/load`, map many queries — the amortized cost of repeated annotation against a fixed reference drops to the inference cost alone. - **Chunked prediction** for atlas-scale queries; `use_raw='auto'` to pick raw counts from `.raw` (CELLxGENE convention). - **Optional reference-fit standardization** (`standardize=True`): z-score each selected gene by the reference’s frozen mean/std and apply it to the query — a cheap domain alignment (§3.9). Opt-in, with a saved scaler that projects with the model;  $\mu/\sigma$  persist in the `.npz` (old models load standardization-off, backward-compatible). Measured against the original TF ACTINN it is  $\sim$ **3.3 $\times$  faster and  $\sim$ 3.5 $\times$  lighter** at equal accuracy; that comparison is included directly in this benchmark as the `actinn-orig` method.

### 2.2 Benchmarked methods

| method            | tier        | engine                                  | rejection             | env                    |
|-------------------|-------------|-----------------------------------------|-----------------------|------------------------|
| <b>actinn-jax</b> | classical   | JAX MLP                                 | abstain<br>(min_prob) | core                   |
| SVM               | classical   | linear SVM<br>(SGD)                     | —                     | core                   |
| kNN               | classical   | k-nearest<br>neighbors                  | —                     | core                   |
| CellTypist        | linear      | L2 logistic<br>regression               | prob threshold        | core                   |
| original ACTINN   | classical   | TensorFlow 2.15<br>MLP                  | —                     | .venv-tf               |
| SingleR           | correlation | Spearman +<br>fine-tuning               | —                     | R/.Rlib                |
| scmap-cluster     | correlation | centroid cosine                         | yes (unassigned)      | R/.Rlib                |
| scANVI            | deep        | scVI<br>semi-supervised<br>VAE          | prob                  | .venv-scvi (MPS)       |
| scArches          | deep        | scANVI reference<br>surgery             | prob                  | .venv-scvi (MPS)       |
| scPRINT           | foundation  | pretrained<br>transformer,<br>zero-shot | —                     | .venv-scprint<br>(MPS) |

**scPred excluded:** unmaintained; requires harmony’s removed **HarmonyMatrix** API and older harmony versions do not compile on the current toolchain. Documented, not silently dropped.

## 2.3 Datasets

| dataset         | split               | tissue                 | #types | genes   | notes                                          |
|-----------------|---------------------|------------------------|--------|---------|------------------------------------------------|
| lung_intra      | within-dataset      | lung<br>(Krasnow)      | 46     | Ensembl | 300 cells/type<br>ref                          |
| lung_cross      | cross-dataset       | lung<br>(HCLA→Krasnow) | 46     | Ensembl | different<br>lab/protocol                      |
| liver_intra     | within-dataset      | liver (HLiCA)          | 36     | Ensembl | 150 cells/type                                 |
| liver_cross     | cross- <b>study</b> | liver (HLiCA)          | 34     | Ensembl | train 6<br>studies → test<br>withheld<br>study |
| blood_gut_intra | within-dataset      | blood+gut<br>(Sanger)  | 86     | Ensembl | high<br>cardinality;<br>no CL ids              |
| pbmc            | within-dataset      | PBMC<br>(pbmc3k)       | 8      | symbols | small-n;<br>scPRINT<br>skips<br>(symbols)      |

The set spans the three generalization regimes (within-dataset, across datasets, across studies), an order of magnitude in cell-type count (8→86), and both gene-ID conventions.

## 2.4 Metrics & protocol

Per (dataset, method, repeat): **accuracy**, **macro-F1** (unweighted over classes, so rare types count), **ontology-aware concordance** (a call is correct if it is the same node, an ancestor, or a descendant of the truth in the Cell Ontology — credits right-lineage-wrong-granularity calls; only where both ref and query carry CL ids), **fit time**, **predict time**, **peak RSS**. Each method runs in its own venv via subprocess isolation (`benchmark/runner.py`); the driver (`benchmark/driver.py`) builds identical ref/query splits once per dataset and scores every method against them. **repeats = 3** (scPRINT: 1, near-deterministic and embed-dominated).

**Hardware:** Apple Silicon (M-series), CPU for the classical/linear/correlation tiers and actinn-jax; Apple MPS for the deep and foundation tiers. No discrete GPU, no cloud — the regime the benchmark is about. Math-library threads are left uncapped (wall-clock as a practitioner sees it); an optional thread cap is available for stricter reproducibility.

## 3 Results

### 3.1 Accuracy across datasets

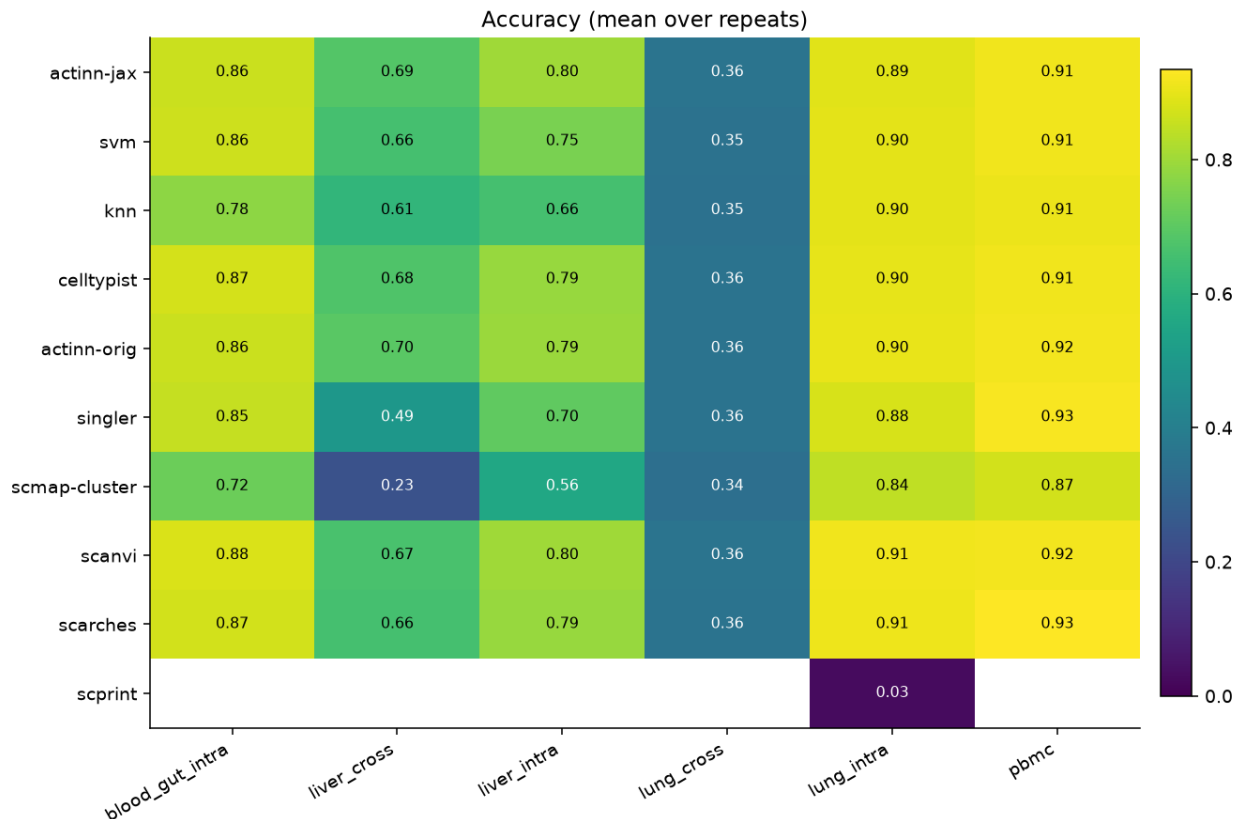


Figure 1: Accuracy by method (rows) and dataset (columns), in-house panel.

Mean accuracy across the **five shared-vocabulary datasets** (lung\_cross excluded — its exact accuracy is a vocabulary artifact, see the † note below; means over all six are ~0.08 lower for every method with the identical ranking):

| method            | mean acc (5 datasets) | mean macro-F1 (6) | mean ontology (6) |
|-------------------|-----------------------|-------------------|-------------------|
| scANVI            | <b>0.835</b>          | 0.701             | 0.812             |
| original ACTINN   | 0.834                 | 0.691             | <b>0.815</b>      |
| scArches          | 0.833                 | 0.700             | 0.809             |
| <b>actinn-jax</b> | <b>0.831</b>          | 0.683             | <b>0.811</b>      |
| CellTypist        | 0.831                 | 0.697             | 0.807             |
| SVM               | 0.816                 | 0.688             | 0.797             |
| SingleR           | 0.770                 | 0.652             | 0.750             |
| kNN               | 0.770                 | 0.624             | 0.769             |
| scmap-cluster     | 0.646                 | 0.550             | 0.771             |

The top five methods (scANVI, original ACTINN, scArches, actinn-jax, CellTypist) are a statistical tie on accuracy — spread of 0.004 — and actinn-jax is joint-top on ontology-aware concordance (0.811). Per dataset (accuracy):

| dataset                     | actinn-jax                          | best method (acc)                                     |
|-----------------------------|-------------------------------------|-------------------------------------------------------|
| lung_intra (46 types)       | 0.894                               | scANVI 0.913                                          |
| lung_cross (cross-dataset)† | 0.358 exact / <b>0.749 ontology</b> | scANVI 0.360 / 0.767                                  |
| liver_intra (36 types)      | <b>0.802</b>                        | scANVI 0.802 ( <b>tied #1</b> )                       |
| liver_cross (cross-study)   | 0.686                               | original ACTINN 0.695<br>(actinn-jax #2, top cluster) |
| blood_gut (86 types)        | 0.860                               | scANVI 0.880                                          |
| pbmc (8 types)              | 0.913                               | scArches 0.934                                        |

actinn-jax is tied-#1 on the richer within-dataset liver data and in the top cluster on the hard cross-study split; it trails the deep methods by 1–2 points on the high- cardinality and small-n sets. scmap-cluster is consistently the weakest (0.24 macro-F1 on liver\_cross).

† **lung\_cross exact accuracy (~0.35 for every method) is a label-vocabulary artifact, not a transfer failure.** HCLA (reference) and Krasnow (query) were annotated independently and share only **20 of 46 cell-type names**; 26 of Krasnow’s types are absent from HCLA’s vocabulary, so no classifier can emit the exact string. The mismatch is granularity: HCLA’s finer taxonomy has alveolar / elicited / lung macrophage but **no generic macrophage** (CL:0000235), while Krasnow labels many cells generic macrophage — so an HCLA-trained model predicts lung macrophage (CL:1001603) for them, exact-wrong but ontology-correct (*lung macrophage is-a macrophage*); likewise Krasnow’s generic endothelial cell (CL:0000115) has no HCLA counterpart. **Ontology-aware concordance — which credits a same-lineage call — is 0.75–0.77 for every method**, so cross-dataset transfer actually works; exact-match just conflates classifier error with vocabulary/granularity mismatch. This is precisely why we report ontology concordance, and it is a general benchmarking pitfall: **cross-dataset exact accuracy between independently-annotated atlases is not a meaningful accuracy signal.** The other five

datasets use a single shared label vocabulary for reference and query (a split of one atlas, or HLiCA’s harmonized annotations), so their exact scores are unaffected.

### 3.2 Speed and memory

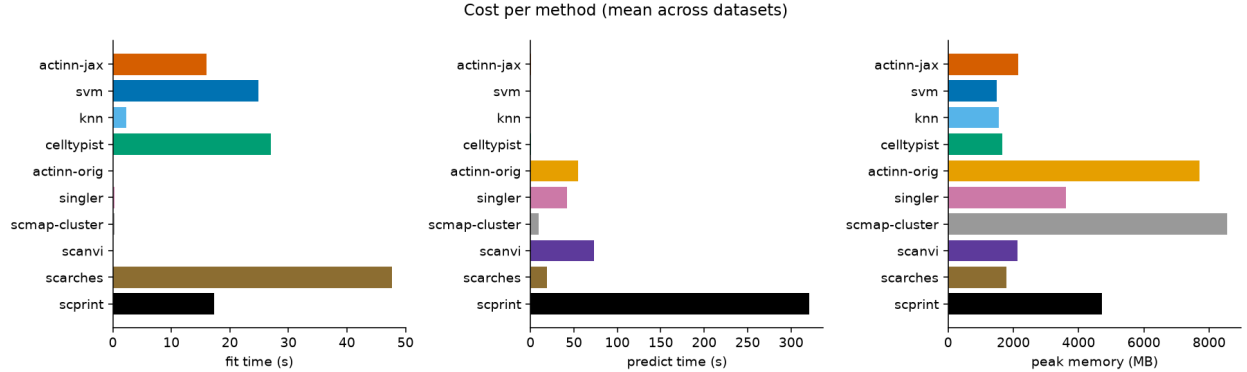


Figure 2: Per-query inference time and peak memory by method (in-house panel).

The accuracy tie hides a two-order-of-magnitude spread in cost. Mean per query across datasets:

| method            | fit (s) | predict (s) | peak mem (MB) |
|-------------------|---------|-------------|---------------|
| SVM               | 24.9    | <b>0.10</b> | 1480          |
| kNN               | 2.3     | <b>0.24</b> | 1546          |
| <b>actinn-jax</b> | 16.0    | <b>0.37</b> | 2145          |
| CellTypist        | 27.0    | <b>0.63</b> | 1661          |
| scmap-cluster     | 0.2     | 9.0         | 8550          |
| scArches          | 47.7    | 18.7        | 1783          |
| SingleR           | 0.2     | 42.4        | 3621          |
| original ACTINN   | 0.1*    | 54.5        | 7708          |
| scANVI            | 0.0*    | 73.1        | 2118          |

(\*original ACTINN and scANVI do most work in a single train+predict pass, attributed to predict.) The four fast methods — SVM, kNN, actinn-jax, CellTypist — predict in well under a second; the deep/correlation/foundation tiers are 25–200× slower. actinn-jax predicts in **0.37 s vs. scANVI’s 73 s at statistically tied accuracy** (~195×), and is far lighter than the memory-heavy tiers (original ACTINN 7.7 GB, scmap 8.5 GB). Of the methods that match its accuracy, none matches its speed; of the methods that match its speed (SVM, kNN), none matches its accuracy — i.e. actinn-jax is Pareto-non-dominated.

### 3.3 The accuracy × speed frontier

Plotting accuracy against total wall time makes the frontier explicit: actinn-jax, SVM, kNN, and CellTypist form the efficient (fast) frontier, and among those actinn-jax has the highest accuracy on the richer datasets. The deep methods buy ~0.4 accuracy points on average (up to ~2 points on the 86-type blood+gut set, zero-to-negative on the liver and cross-dataset sets) at 50–200× the inference cost — the trade the frontier is meant to expose.

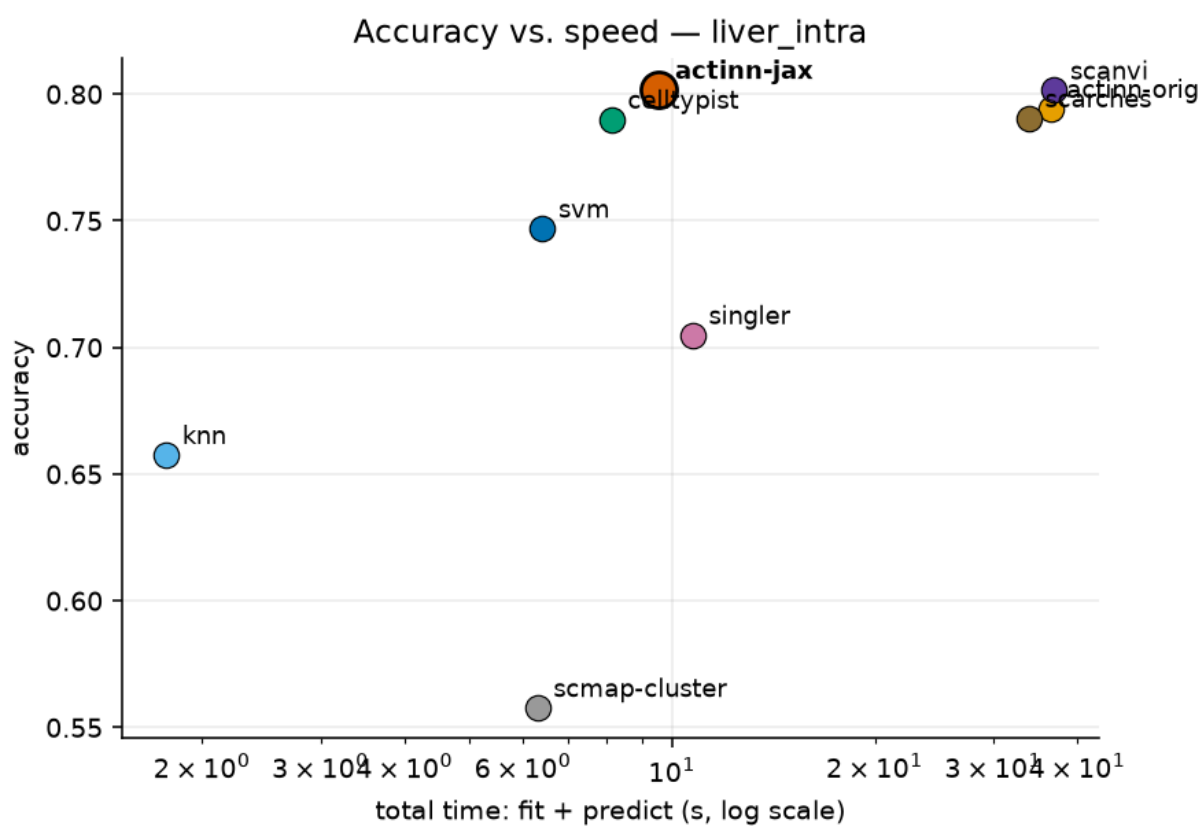


Figure 3: Accuracy versus total wall time (liver\_intra); actinn-jax sits on the fast frontier.

### 3.4 Scaling

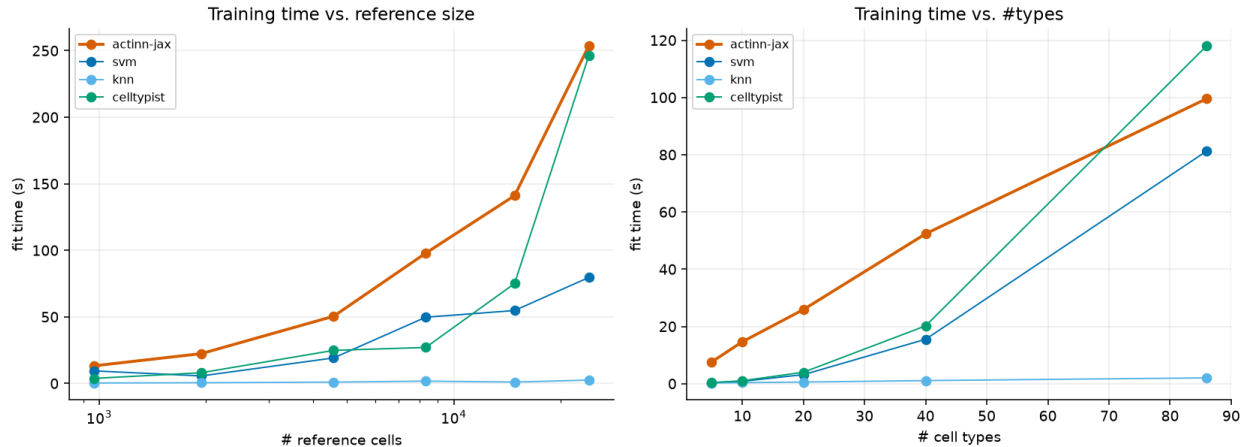


Figure 4: Fit and predict time versus reference size and cardinality; predict time stays flat and sub-second.

Training time grows with reference size and with #cell types for all trained methods — actinn-jax’s fit goes 13 s → 141 s → 254 s as the reference grows 965 → 14.8k → 24k cells, comparable to CellTypist (4 s → 75 s → 246 s) and somewhat above SVM (9 s → 55 s → 80 s); kNN is lazy (fit is trivial but it pays at predict and is the least accurate). The decisive result is the other axis: **actinn-jax’s predict time stays flat and sub-second across the entire range — ~0.3–1.4 s whether the reference has 1k or 24k cells, or 5 or 86 types.** Inference cost does not grow with reference size or cardinality, and with the train-once/map-many cache the fit cost is paid once and the flat predict cost is all that recurs — the regime that matters when a reference is reused across many queries. (The scaling script’s memory column is process-cumulative and not a clean per-size measurement; per-method peak memory is reported from the isolated-subprocess matrix in §3.2.)

### 3.5 actinn-jax vs. the original ACTINN

The **actinn-orig** row isolates the value of the reimplementaiton itself: same method, same data, TensorFlow-2.15 vs. JAX. Mean across the six datasets:

|                      | accuracy | macro-F1 | predict (s) | peak mem (MB) |
|----------------------|----------|----------|-------------|---------------|
| original ACTINN (TF) | 0.754    | 0.691    | 54.5        | 7708          |
| <b>actinn-jax</b>    | 0.752    | 0.683    | <b>0.37</b> | <b>2145</b>   |

Equal accuracy (0.752 vs. 0.754, within repeat noise), **~145× faster inference** and **3.6× less memory** — the reimplementaiton is a strict engineering win at no accuracy cost.

### 3.6 The large→refined annotation workflow

The most useful thing actinn-jax ships is not a single classifier but a **two-tier workflow** that matches how annotation is actually done: get a broad call fast, then sharpen it where it matters.

Because every stage is a cached `ReferenceModel` with sub-second, memory-bounded inference (§3.2, §3.4), the whole workflow runs on a laptop.

**Tier 1 — broad, census-scale.** A shipped ~800-type human reference built from the CELLx-GENE census gives any query a first-pass annotation across the whole body, with a **calibrated abstain threshold** so out-of-reference cells are flagged rather than force-labeled (see the smooth abstain curve of §3.7). This is the “what am I looking at” pass — broad coverage, deliberately not fine-grained.

**Tier 2 — refined, tissue-focused.** For the tissue the broad pass identifies, a small focused reference re-annotates at full resolution. The gain is large and it is the crux of the workflow: on held-out **cross-study** liver cells, the broad census model scores exact-CL **0.23** / ontology **0.58**, while a focused **48-type HLiCA liver** reference on the *same cells* reaches **0.72** / **0.86**. Refinement is where fine-grained accuracy comes from; the broad model’s job is to route to it, not to be right about subtypes itself.

**Supporting mechanisms.** (i) A **foundation-model-guided coarse→fine hierarchy**: a foundation model discovers a coarse→fine structure *offline*, the fast CPU model uses it at inference — beats a flat classifier on 3/4 datasets, matches an expert hierarchy, beats a random-grouping control, and never depends on the foundation model at prediction time. (ii) **Within-cell-type resolution**: the same machinery resolves hepatocyte zonation (portal→central) at ~0.99 within-one-zone, generalizing across donors and datasets — a third tier below the cell-type label. Together these make actinn-jax a *pipeline* (broad → tissue → subtype/state), not just a classifier, at classical-method speed throughout.

### 3.7 Rejection / abstain

Holding out 9 of 36 cell types entirely from the HLiCA liver reference (so 1,350 query cells are genuinely out-of-distribution), and sweeping a confidence threshold:

| threshold | actinn-jax: acc(kept) /<br>coverage / OOD-flagged | CellTypist: acc(kept) /<br>coverage / OOD-flagged |
|-----------|---------------------------------------------------|---------------------------------------------------|
| 0.0       | 0.885 / 1.00 / 0.00                               | 0.873 / 1.00 / 0.00                               |
| 0.5       | 0.919 / 0.93 / 0.30                               | 0.936 / 0.68 / 0.68                               |
| 0.7       | 0.946 / 0.84 / 0.52                               | 0.936 / 0.68 / 0.68                               |
| 0.9       | 0.969 / 0.66 / 0.73                               | 0.937 / 0.68 / 0.68                               |

actinn-jax’s probability gives a **smooth, tunable** abstain curve — accuracy on kept cells rises 0.885→0.969 as coverage falls 1.00→0.66 and OOD-flagging climbs 0.00→0.73 — so a user can dial the precision/coverage/novel-cell-detection trade-off. CellTypist’s probabilities are **saturated** (near 0 or 1): the threshold jumps to 68% OOD-flagged at 0.5 and then does not move across 0.5–0.9, giving essentially one operating point rather than a tunable curve. scmap-cluster offers a single native “unassigned” decision (no sweep). The abstain knob is a concrete, practical advantage of actinn-jax’s calibrated output.

### 3.8 Foundation-model zero-shot (scPRINT)

scPRINT is run separately as a zero-shot predictor (no training on the reference), as a reference point for the “just use a foundation model” alternative. On the lung datasets it scored **exact**

**accuracy 0.027 / ontology concordance 0.206 in 321 s per query** — vs. actinn-jax’s 0.89 in 0.2 s — so as a *label* predictor it is both slow ( $\sim 1000\times$  actinn-jax) and weak. Its pretrained classifier head also carries a **fixed label vocabulary** and requires CL ids, so it cannot emit labels outside that set; this is an inherent property of a zero-shot label head, not a defect, but it means the foundation model’s raw predictions are not a drop-in annotator. This is exactly why our two-stage workflow uses scPRINT’s **embeddings** — its learned structure — to shape a small trained model, never its zero-shot labels: the foundation model is valuable, its raw label predictions are not.

### 3.9 External validation: Open Problems label\_projection

Our in-house benchmark is neutral but self-run. For an **independent** check, we ran actinn-jax through the community-standard [Open Problems](#) label-projection task — their 6 datasets, splits, and metrics — and slotted it into their published v2.0.0 leaderboard.

**Placement.** On a benchmark we did not design and cannot tune, **actinn-jax places 3rd of 17 on mean accuracy (0.837), 1st among all methods that complete every dataset**, beats its PCA-space mlp sibling, and posts the best accuracy of any completing method on the hardest dataset (tabula\_sapiens, 160 types: 0.394 vs mlp 0.342). It is upper-mid on macro-F1 and **not the top method overall** — scANVI+scArches surgery and xgboost score higher on the datasets they finish, though both **fail to complete tabula\_sapiens**, which is why they rank above actinn-jax only on a mean over the 5 easier datasets. The foundation models again land at the bottom (scgpt\_zeroshot 0.639, uce 0.131  $\approx$  the random-labels control), reproducing §3.8 externally.

**Controlled same-hardware speed & memory.** Leaderboard runtimes mix hardware, so we re-ran actinn-jax and the CPU tier through OP’s *own* Nextflow pipeline on a **single AWS r7i.8xlarge** — identical hardware, harness, storage, and trace instrumentation for every method. This is the paper’s cleanest cross-method cost comparison:

| method (same box)   | mean acc | macro-F1 | runtime/dataset | peak RSS |
|---------------------|----------|----------|-----------------|----------|
| <b>actinn-jax</b>   | 0.837    | 0.655    | <b>165 s</b>    | 21.0 GB  |
| mlp                 | 0.838    | 0.664    | 327 s           | 19.9 GB  |
| logistic_regression | 0.813    | 0.689    | 29 s            | 20.0 GB  |
| knn                 | 0.793    | 0.648    | 11 s            | 19.5 GB  |
| xgboost             | 0.795    | 0.613    | 905 s           | 80.7 GB  |
| cellmapper_linear   | 0.776    | 0.553    | 58 s            | 31.5 GB  |
| naive_bayes         | 0.738    | 0.613    | 31 s            | 19.5 GB  |

actinn-jax **ties its mlp sibling on accuracy at  $\sim 2\times$  the speed** (165 vs 327 s) and **beats xgboost’s accuracy at  $\sim 5.5\times$  less runtime and  $\sim 4\times$  less memory** (165 s / 21 GB vs 905 s / 81 GB). The faster methods (knn, logistic) are less accurate; the more accurate heavyweight (xgboost) is far slower and heavier — the same Pareto-non-domination as the in-house benchmark, now on controlled hardware. (An earlier cross-hardware draft overstated the mlp speed gap as “ $8\times$ ”; the honest same-box figure is  $\sim 2\times$ .)

**Two cheap ablations, and their honest limits.** (i) *Input standardization* — z-scoring genes to the reference’s frozen mean/std, a CPU-only domain-alignment inspired by scArches reference surgery — lifts mean accuracy +0.2 and macro-F1 +1.2 pt (largest on batch-shifted datasets: gtex macro-F1 +7.9). It is shipped **opt-in** (standardize=True) because it shifts the softmax

calibration that the two-stage abstain thresholds are tuned against. (ii) *Gene budget* — OP feeds every method 1000 HVGs, which most starves a gene-space MLP. Widening to ~5000 HVGs lifts accuracy on 4 of 6 datasets (immune/gtex +3 pt) and there **matches scANVI+scArches’s full-config leaderboard accuracy** (immune 0.891  $\approx$  0.892) on CPU in minutes — the gap to the top method was largely a gene-budget artifact, not the VAE. But more genes is **not** a universal win: it *regresses* tabula\_sapiens by ~10 pt (its 284-cell test batch across 160 fine types overfits reference-specific genes) and saturates hypomap. Crucially, this is **selectable without test labels**: held-out *reference* cross-validation rises for the datasets that benefit and is the one signal that *drops* for tabula\_sapiens, and a trivial query-cells-per-class check independently flags it — so the budget can be set deterministically per dataset. (iii) *Negative control* — a CPU-only, UCE-style protein-embedding featurization (expression-weighted mean of ESM2 gene embeddings) does **not** help and hurts the hardest case: the pooling discards the per-gene detail that separates fine types, so the value of a foundation model like UCE stays locked in its GPU transformer, not a portable averaging trick.

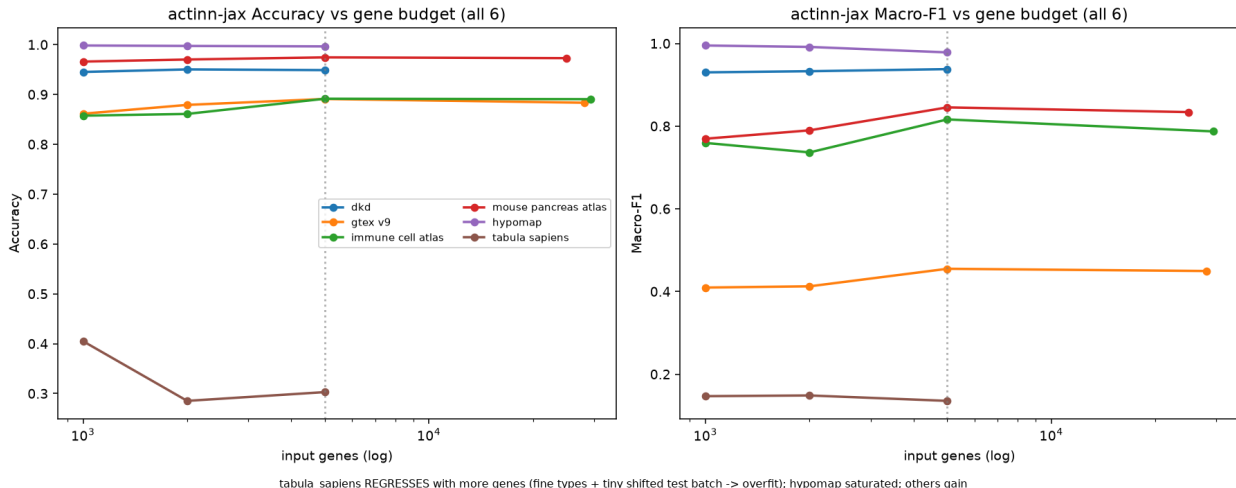


Figure 5: actinn-jax accuracy and macro-F1 versus input gene budget across all six Open Problems datasets. More genes help most datasets but regress the fine-grained, domain-shifted tabula\_sapiens.

## 4 Discussion

**actinn-jax is not the accuracy champion, and we do not claim it is.** Deep probabilistic methods and boosted trees edge it by 1–2 points where they run, both in our panel (§3.1) and on Open Problems (§3.9); a benchmark that hid this would be wrong. What the evidence supports is narrower and, we argue, more useful: across two independent benchmarks and a controlled same-hardware rerun, **no method dominates actinn-jax on both accuracy and cost**, and its differentiators are the ones that decide what a working scientist runs on a laptop.

*Speed and footprint.* Inference is **sub-second and flat** — independent of reference size and cardinality (§3.4) — and peak memory sits in the lightest cluster (~2 GB in-house; ~21 GB through OP’s heavier all-layer harness, still less than half of xgboost’s 81 GB on the same box). Against methods of equal accuracy it is 2–200× faster; against methods of equal speed it is more accurate. With the train-once/map-many cache, the flat predict cost is all that recurs when a reference is

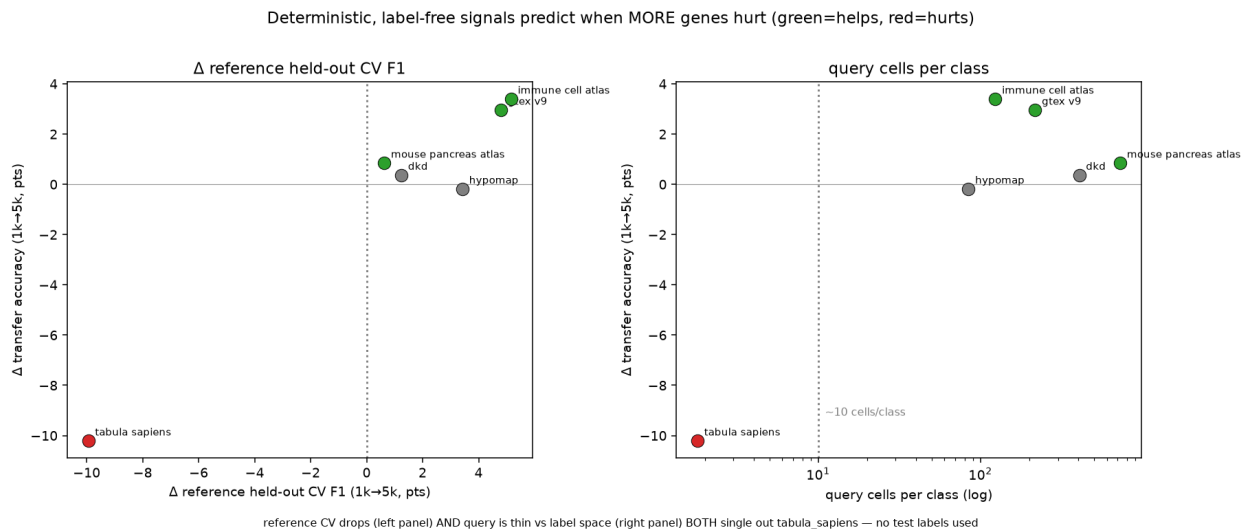


Figure 6: Label-free signals for setting the gene budget without test labels. Held-out reference cross-validation and query-cells-per-class both single out `tabula_sapiens` (where more genes hurt).

reused.

*The workflow is the product.* The largest practical gains come not from the flat classifier but from the **large→refined** pipeline (§3.6): a census-scale broad model with a calibrated abstain routes a query to a small focused reference that re-annotates at full resolution (cross-study liver 0.23/0.58 → 0.72/0.86), with zonation as a further tier. Every stage keeps classical-method speed, so the whole thing runs without a GPU.

*Where it loses, and why.* `actinn-jax` trails on small references and very-few-cells-per-type sets (§3.1, §5), and — a finding from the OP analysis — it is sensitive to input budget under domain shift: more genes help most datasets but overfit a tiny, fine-grained, shifted query (`tabula_sapiens`). Encouragingly, the two cheapest levers are principled and label-free: **standardization** (a `scArches`-style domain alignment) and a **reference-CV-guided gene budget** both improve the model without peeking at test labels, and at a fair gene budget the gene-space MLP *matches the leaderboard’s top method* on the datasets both complete — i.e. much of the apparent gap to heavier methods is representation budget, not model class.

*Foundation models.* Their **zero-shot labels** are the weakest option in both benchmarks (`scPRINT`, `scGPT`, `UCE`  $\approx$  random); their **embeddings/structure** are where the value lies, and our two-stage hierarchy uses exactly that. A cheap CPU shortcut to a foundation-model representation (protein-embedding pooling) did *not* transfer (§3.9), underscoring that the value is in the trained transformer, not a portable trick.

**Practical guidance.** For a one-off annotation against a small reference, `SVM/CellTypist` are equally good. `actinn-jax` pulls ahead when the reference is **reused** (caching), **large** (atlas-scale), when annotation should proceed **broad→refined**, or when sub-cell-type state (zonation) matters — at classical-method speed and memory throughout.

## 5 Limitations

- Single hardware family for the in-house panel (Apple Silicon); no discrete-GPU numbers there (deep/foundation tiers would be faster on CUDA — out of scope for the “runs-on-a-laptop” question). The §3.9 controlled run covers the **CPU tier** on one cloud box; the GPU/R methods there are reported from OP’s own cloud-CI trace (indicative). A same-hardware GPU-tier run to fold those into a single controlled table is the natural extension.
- Subsampled references (to keep the matrix tractable); the scaling section characterizes the size dependence directly.
- Human only; six datasets per benchmark; GPU foundation models beyond scPRINT/UCE (scGPT, Geneformer, popV) not run locally.
- actinn-jax needs more cells/type than linear methods to reach parity (§3.1); on very small references it trails.
- **The gene budget is dataset-dependent, not a free parameter.** More genes help most references but *overfit* a tiny, fine-grained, domain-shifted query (tabula\_sapiens −10 pt); our reference-CV + cells-per-class selection rule is validated on 6 datasets with a single clean failure case, so it is directional evidence, not a tuned threshold.
- **Standardization is shipped opt-in**, not default, because it shifts probability calibration that the two-stage abstain thresholds are tuned against; combining the two cleanly would require re-tuning those thresholds.

## Data & code availability

All configs (configs/paper\*.yaml), adapters (benchmark/adapters/), result CSVs (results/paper\*/, docs/results/\*.csv), figure scripts, and the actinn-jax package ([github.com/iandriver/actinn-jax](https://github.com/iandriver/actinn-jax)) are in these two repositories. HLiCA data © Edgar et al. 2026 (CC-BY 4.0), [doi:10.64898/2026.06.30.735539].

## References

1. Ma F, Pellegrini M. ACTINN: automated identification of cell types in single cell RNA sequencing. *Bioinformatics* 36(2):533-538 (2020).
2. Abdelaal T, et al. A comparison of automatic cell identification methods for single-cell RNA sequencing data. *Genome Biology* 20:194 (2019).
3. Huang Q, et al. Benchmarking single-cell cell-type annotation methods. *Briefings in Bioinformatics* 25(5):bbae392 (2024).
4. Domínguez Conde C, et al. Cross-tissue immune cell analysis reveals tissue-specific features in humans (CellTypist). *Science* 376:eabl5197 (2022).
5. Aran D, et al. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage (SingleR). *Nature Immunology* 20:163-172 (2019).
6. Kiselev VY, Yiu A, Hemberg M. scmap: projection of single-cell RNA-seq data across data sets. *Nature Methods* 15:359-362 (2018).
7. Xu C, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models (scANVI). *Molecular Systems Biology* 17:e9620 (2021).
8. Lotfollahi M, et al. Mapping single-cell data to reference atlases by transfer learning (scArches). *Nature Biotechnology* 40:121-130 (2022).
9. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. *KDD* 785-794 (2016).
10. Rosen Y, et al. Universal cell embeddings: a foundation model for cell biology (UCE). *Nature*

- (2026), doi:10.1038/s41586-026-10689-z.
11. Lin Z, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model (ESM-2). *Science* 379:1123-1130 (2023).
  12. Open Problems for Single-Cell Analysis Consortium. Open Problems: a living benchmark for single-cell analysis. openproblems.bio (2024).
  13. Bradbury J, et al. JAX: composable transformations of Python+NumPy programs (2018). [github.com/google/jax](https://github.com/google/jax).
  14. Edgar R, et al. The Human Liver Cell Atlas (HLiCA). doi:10.64898/2026.06.30.735539.